

Contents

- Contents..... 1
- 1. Introduction & Objectives..... 2
- 2. Data Collection.....2
- 3. Data Annotation (Labeling)..... 3
- 4. Data Preprocessing & Format Conversion (Labelme to YOLO)..... 5
- 5. Data Augmentation Pipeline..... 6
- 6. Cloud Environment Setup (Google Colab).....8
- 7. Dataset Configuration Update..... 8
- 8. Model Architecture & Hyperparameters..... 9
- 9. Training Execution & Convergence..... 10
- 10. Model Evaluation & Results..... 11
- 11. Error Analysis & Training Metrics..... 11
- 12. Limitations and Edge Case Analysis..... 15
- 13. Final Output & Conclusion..... 18

Project Report: Automated Data Extraction from Partial Tax Invoices using YOLO

1. Introduction & Objectives

The goal of this project is to build an object detection model capable of reading and extracting specific line-item data from Thai partial tax invoices (receipts). Rather than simply running raw Optical Character Recognition (OCR) on the whole document, I am training a YOLO-based model to locate and classify four specific bounding boxes first:

- `quantity_item`: The number of items purchased
- `product_description`: The name/details of the item (primarily in Thai)
- `price`: The total price for that specific line item
- `total_due_amount`: The final grand total of the receipt

By isolating these regions first, the data pipeline becomes much more structured and accurate.

2. Data Collection

To ensure the model generalizes well, I collected a diverse dataset of roughly 200 receipts from my own personal purchases.

- **Physical Receipts:** The majority of the dataset consists of paper receipts from convenience stores and supermarkets like 7-Eleven and Tops (e.g., Tops Belle Grand Rama 9). Because paper receipts get crumpled or have varying lighting, I used the CamScanner app to digitize them. This ensured the text was relatively flat but still retained real-world artifacts.
- **Digital Receipts:** I also included digital e-receipts (like from Uniqlo) to introduce clean, perfectly aligned text into the training set.

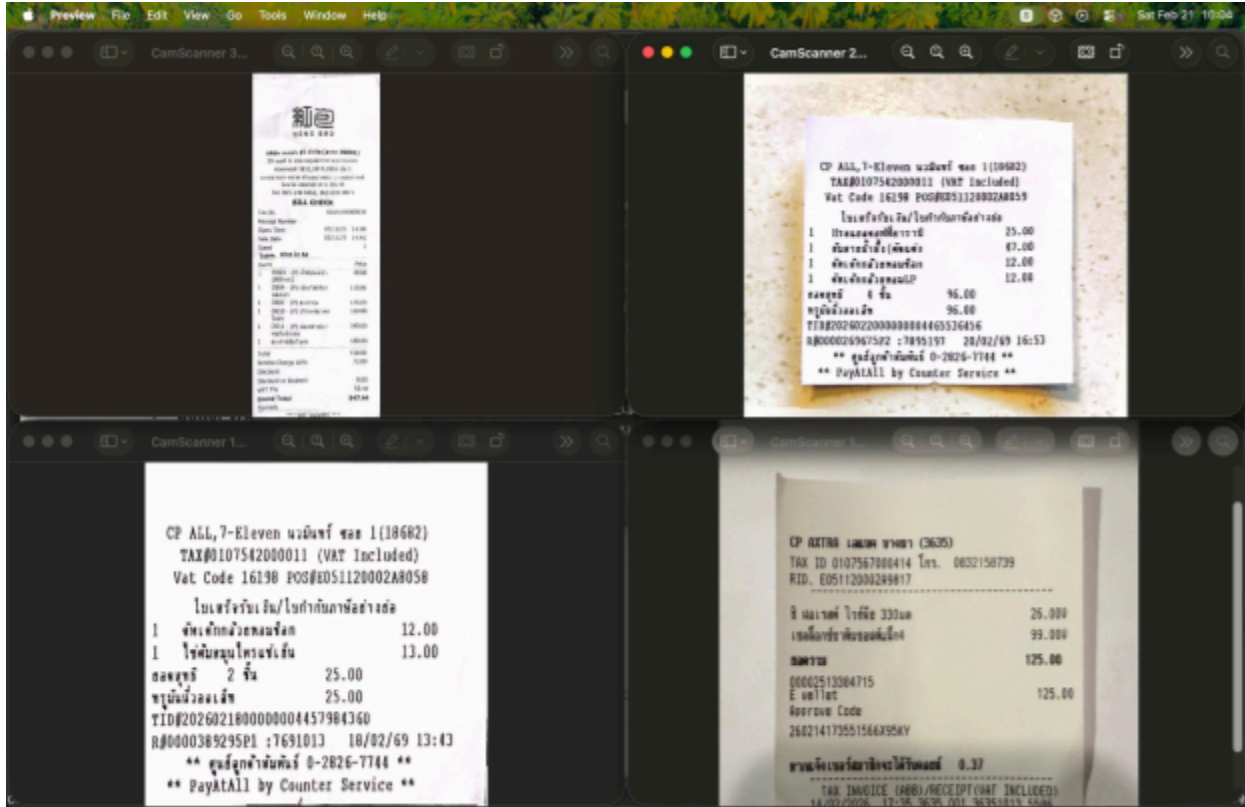


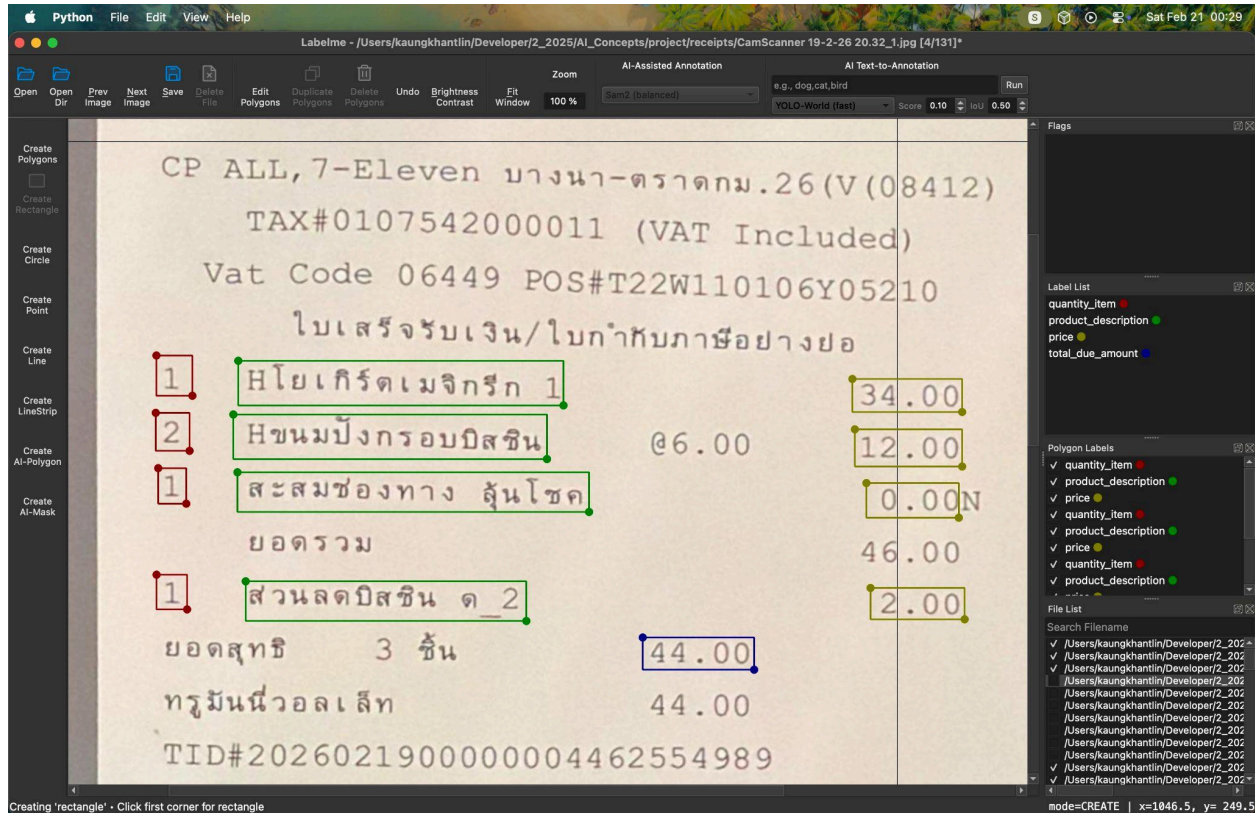
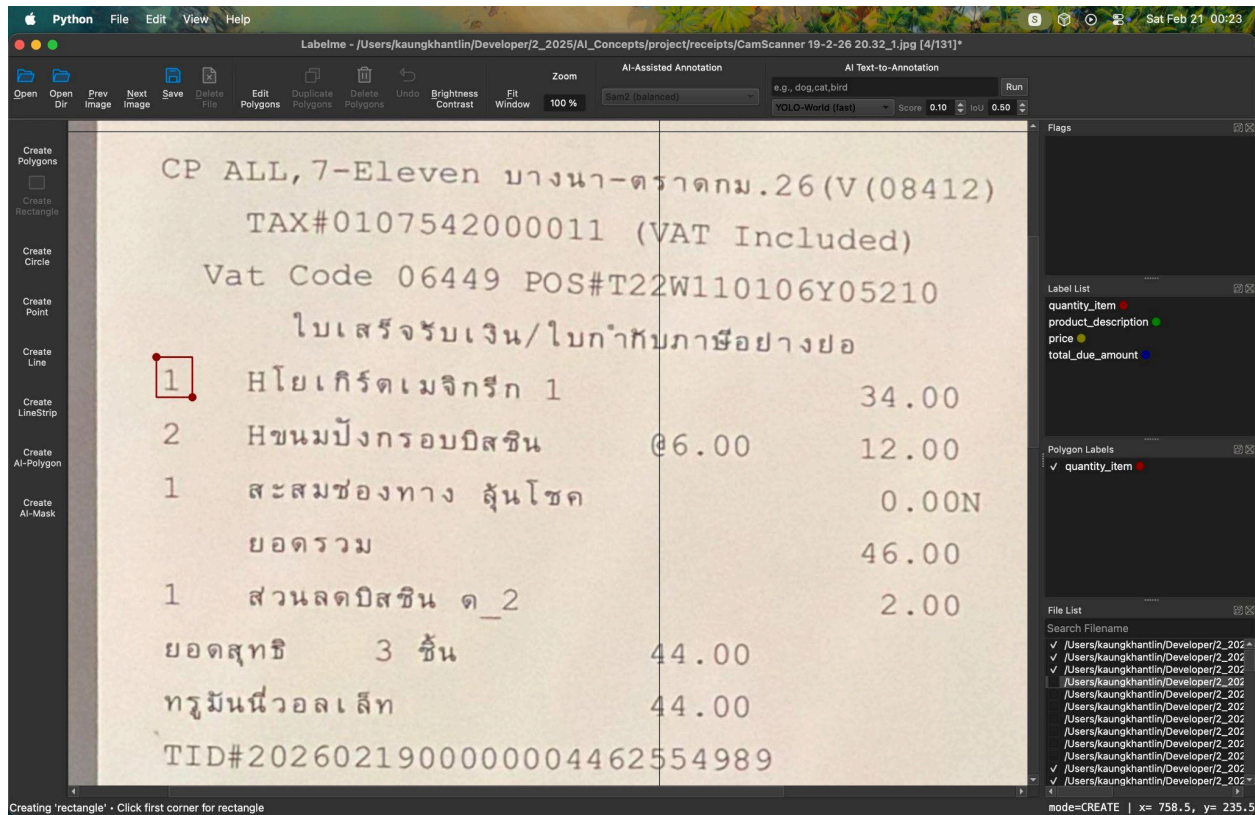
Figure 1: Raw receipts collected from several sources.

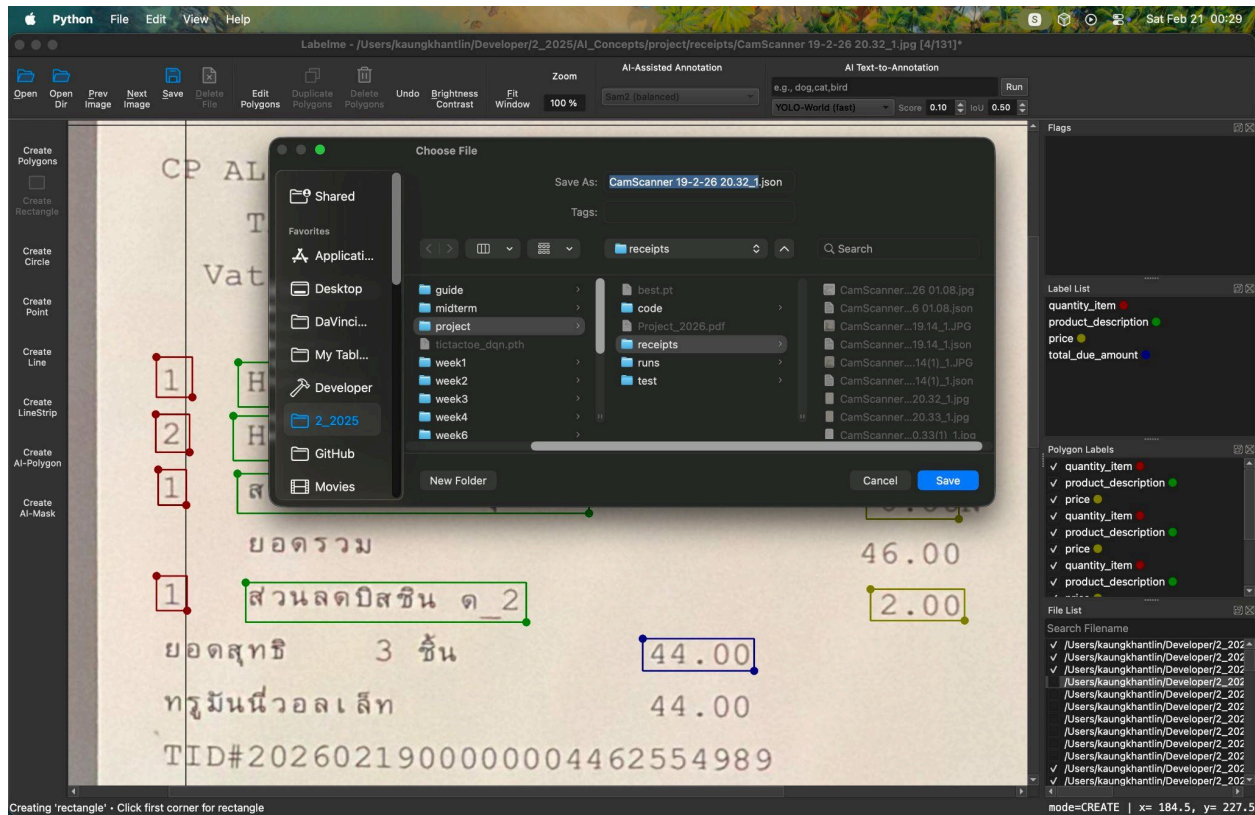
3. Data Annotation (Labeling)

I manually labeled all collected images using the Labelme annotation tool. I saved the resulting .json files in a local folder named receipts alongside the original images.

To ensure consistency across the dataset, I established the following strict labeling rules:

- **Excluding Currency Symbols:** I drew bounding boxes strictly around the numerical values for price and total_due_amount (excluding the "฿" symbol). This prevents the model from getting confused when symbols are missing.
- **Implicit Quantities:** If a receipt did not explicitly state a quantity (implying 1), I skipped the quantity_item label for that row rather than drawing a box over empty space, which would confuse the object detection model.
- **Thai Vowels/Tones:** For the product_description, I ensured the bounding boxes were tall enough to capture upper and lower Thai tonal marks (e.g., ^๕, _๕).





Figures 2-4: Manual labeling in Labelme.

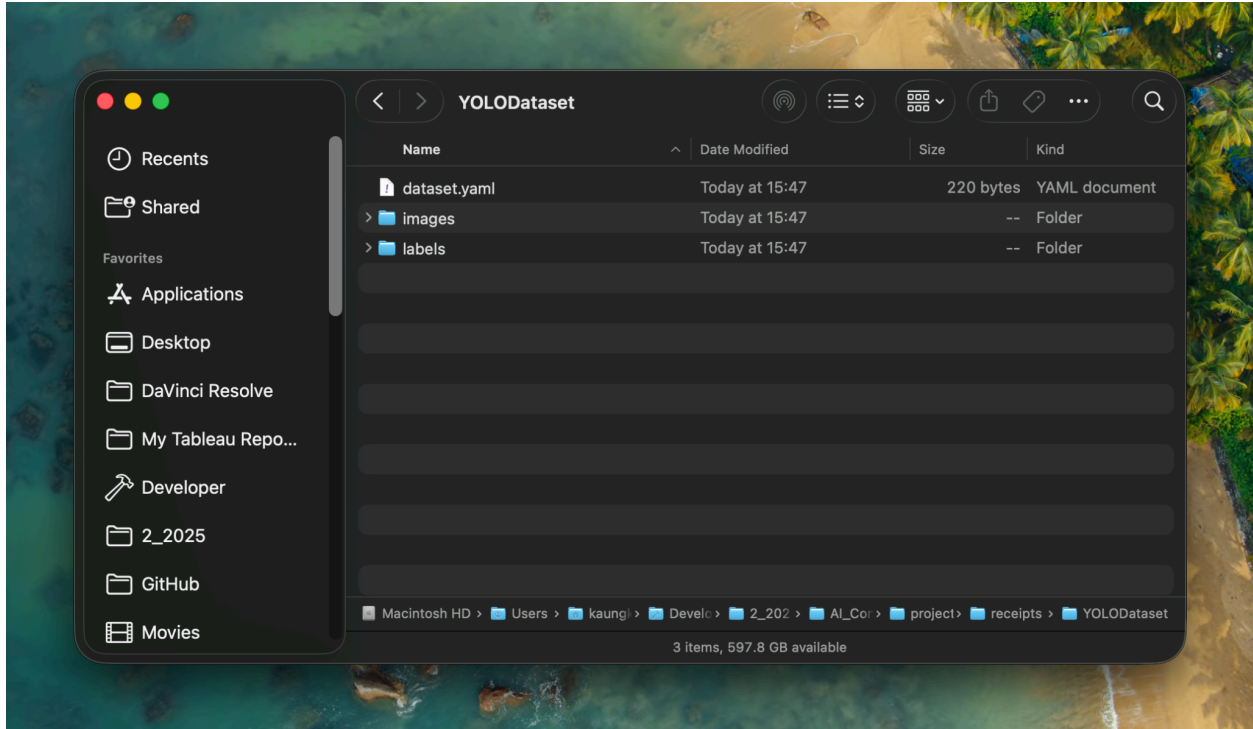
4. Data Preprocessing & Format Conversion (Labelme to YOLO)

Since Labelme outputs annotations in JSON polygon format, I needed to convert them into YOLO's required .txt bounding box format (class_id, x_center, y_center, width, height) before training.

I used the `labelme2yolo` Python package to handle the conversion and automatically split my dataset into Training and Validation sets. I opened my terminal and ran the following command:

```
labelme2yolo --json_dir "receipts" --val_size 0.2 --test_size 0.0
--output_format bbox
```

This successfully generated a `YOLOdataset` folder with an 80/20 train-validation split, ensuring I had untouched data to evaluate the model's true performance later.



5. Data Augmentation Pipeline

Because my initial dataset consisted of 200 images, I needed to artificially expand the training set to prevent the model from overfitting. I wrote a custom Python script (`augment_yolo.py`) using the `albumentations` library to generate 5 augmented copies of every training image.

Important Design Choice: I intentionally avoided spatial augmentations like `HorizontalFlip`. Since receipts follow a strict left-to-right reading logic (Description on the left, Price on the right), flipping the image would destroy the spatial context the model needs to distinguish a price from a quantity.

Instead, my augmentation pipeline focuses on simulating bad camera conditions using:

- `SafeRotate` and `Perspective` to simulate slightly crooked scans.
- `RandomBrightnessContrast`, `RGBShift`, and `HueSaturationValue` to simulate different scanner lights, phone camera flashes, and aged paper.
- `ISONoise` to simulate grainy photos from low-quality cameras.

```

transform = A.Compose(
    [
        # 1. GEOMETRY (Conservative)
        # minimal rotation to simulate slightly crooked scans
        A.SafeRotate(limit=3, p=0.5),

        # random perspective simulates taking a photo at an angle
        A.Perspective(scale=(0.02, 0.05), keep_size=True, p=0.3),

        # 2. PIXEL LEVEL (Aggressive is okay here)
        # Simulates different scanner lights or phone camera flashes
        A.RandomBrightnessContrast(brightness_limit=0.2,
contrast_limit=0.2, p=0.5),

        # Simulates different paper ages (yellowing) or printer ink levels
        A.RGBShift(r_shift_limit=20, g_shift_limit=20, b_shift_limit=20,
p=0.3),
        A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20,
val_shift_limit=20, p=0.3),

        # 3. NOISE (Simulate bad cameras/printing)
        # ISO noise is better than blur for text because edges remain sharp
        A.ISONoise(color_shift=(0.01, 0.05), intensity=(0.1, 0.5), p=0.3),

        # Only use very weak blur if necessary
        A.GaussianBlur(blur_limit=(3, 3), p=0.1),
    ],
    bbox_params=A.BboxParams(format="yolo", min_visibility=0.3,
label_fields=["class_labels"])
)

```

After configuring the paths to point to the output from Step 4, I ran the script, successfully generating a finalized YOLODataset_aug folder ready for model training.

6. Cloud Environment Setup (Google Colab)

Training an object detection model especially a larger architecture like YOLO261 locally on a CPU is highly inefficient. Therefore, I utilized Google Colab to access cloud-based GPU acceleration.

First, I compressed my finalized YOLODataset_aug folder into a .zip file and uploaded it to my Google Drive. Inside my Colab notebook, I mounted my Drive and used the following command to extract the dataset directly into the local /content/ directory of the Colab instance:

```
!unzip -q "/content/drive/MyDrive/Project/YOLODataset_aug.zip" -d /content/
```

Justification: Extracting the files to the local Colab instance rather than reading them directly from Google Drive during training drastically reduces I/O bottlenecks and speeds up the training time per epoch.

I also verified the hardware allocation by running `!nvidia-smi`, which confirmed Google Colab assigned me a Tesla T4 GPU.

7. Dataset Configuration Update

Because the file paths on my local macOS environment differ from the Linux environment in Google Colab, the original dataset.yaml file generated by labelme2yolo would cause "dataset not found" errors.

To solve this, I wrote a Python snippet within the notebook to dynamically read the original YAML file and rewrite it as data_colab.yaml, automatically correcting the path, train, and val directories to point to the Colab environment:

```
import yaml

src = "/content/YOLODataset_aug/dataset.yaml"
with open(src, "r", encoding="utf-8") as f:
    data = yaml.safe_load(f)

# Force correct path for Colab Linux environment
data["path"] = "/content/YOLODataset_aug"
data["train"] = "images/train"
```



```
data["val"] = "images/val"

out = "/content/data_colab.yaml"
with open(out, "w", encoding="utf-8") as f:
    yaml.safe_dump(data, f, sort_keys=False)
```

8. Model Architecture & Hyperparameters

With the dataset structured and the environment ready, I installed the ultralytics package to initialize the YOLO training pipeline.

For the model architecture, I selected yolo26l.pt (the Large variant). While heavier than the nano or small versions, the large model is better suited for extracting fine-grained visual details, such as the small tonal marks in Thai text or distinguishing between a price and a product code.

I executed the training with the following command and hyperparameters:

```
!yolo detect train \
  model=yolo26l.pt \
  data=/content/data_colab.yaml \
  epochs=200 imgsz=640 batch=32 device=0 \
  project="/content/drive/MyDrive/Project/weight" \
  name="yolo26l_custom" \
  save=True save_period=5 \
  mosaic=1.0 close_mosaic=10 \
  flipplr=0.0 \
  hsv_h=0.03 hsv_s=0.6 hsv_v=0.5 \
  degrees=10 translate=0.1 scale=0.5 \
  mixup=0.1
```

Key Hyperparameter Decisions:

epochs=200: Extended the training time to allow the larger model to fully converge on the augmented dataset.

flipplr=0.0: Explicitly disabled horizontal flipping. As established during the offline augmentation phase, flipping a receipt destroys the spatial logic required to identify line items correctly.

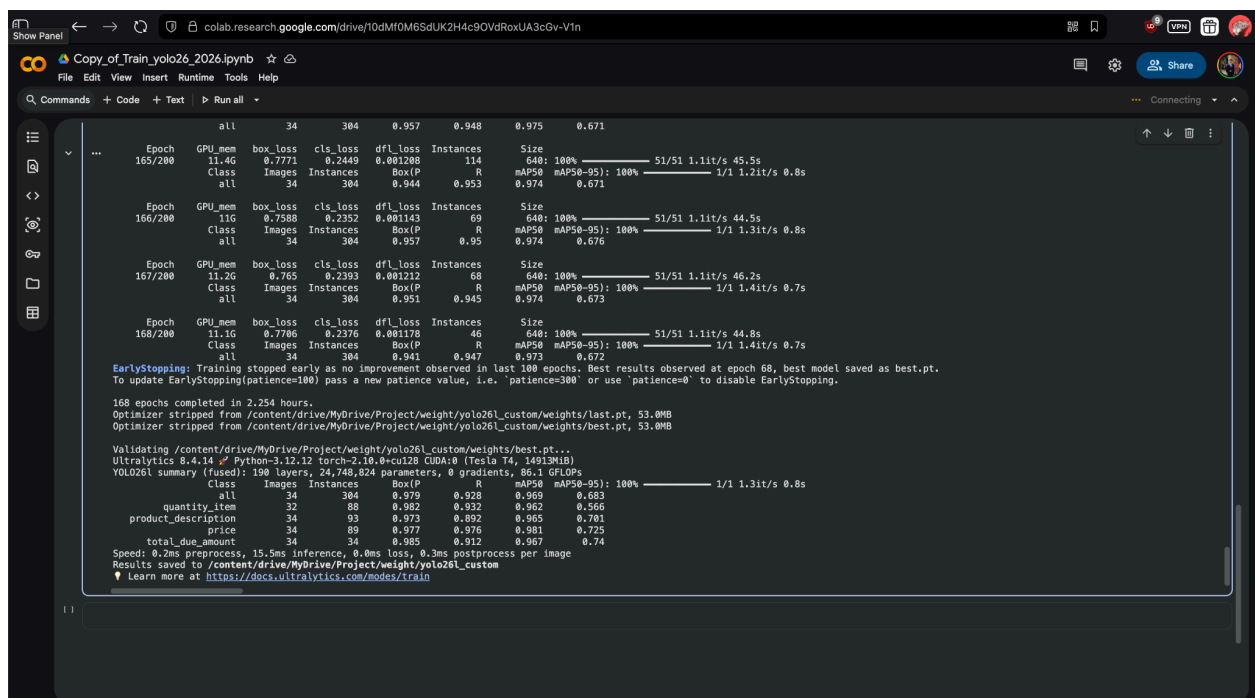
mosaic=1.0: Enabled mosaic augmentation to stitch multiple receipts together during training. This forces the model to learn the contextual relationships between text blocks rather than memorizing their absolute position on the page.

save_period=5: Ensures the model saves checkpoint weights every 5 epochs directly to my Google Drive, preventing data loss if the Colab session disconnects.

9. Training Execution & Convergence

Once the training was initialized, the YOLO framework processed the augmented dataset using Google Colab's Tesla T4 GPU.

While I set the maximum epochs to 200, the Ultralytics framework utilizes a built-in Early Stopping mechanism to prevent overfitting. The training automatically halted after 168 epochs (taking approximately 2.254 hours). The framework detected that the model's performance had peaked earlier and correctly identified that the weights from Epoch 68 provided the highest validation accuracy.



```
Copy of Train_yolo26_2026.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all +
...
Epoch 165/200 GPU_mem 11.4G box_loss 0.7771 cls_loss 0.2449 dfl_loss 0.001208 Instances 114 Size 640: 100% 51/51 1.1it/s 45.5s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.2it/s 0.8s
all 34 304 0.944 0.953 0.974 0.671

Epoch 166/200 GPU_mem 11G box_loss 0.7588 cls_loss 0.2352 dfl_loss 0.001143 Instances 69 Size 640: 100% 51/51 1.1it/s 44.5s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.3it/s 0.8s
all 34 304 0.957 0.95 0.974 0.676

Epoch 167/200 GPU_mem 11.2G box_loss 0.765 cls_loss 0.2393 dfl_loss 0.001212 Instances 68 Size 640: 100% 51/51 1.1it/s 46.2s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.4it/s 0.7s
all 34 304 0.951 0.945 0.974 0.673

Epoch 168/200 GPU_mem 11.1G box_loss 0.7706 cls_loss 0.2376 dfl_loss 0.001178 Instances 46 Size 640: 100% 51/51 1.1it/s 44.8s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.4it/s 0.7s
all 34 304 0.941 0.947 0.973 0.672

EarlyStopping: Training stopped early as no improvement observed in last 100 epochs. Best results observed at epoch 68, best model saved as best.pt.
To update EarlyStopping(patience=100) pass a new patience value, i.e. 'patience=300' or use 'patience=0' to disable EarlyStopping.

168 epochs completed in 2.254 hours.
Optimizer stripped from /content/drive/MyDrive/Project/weight/yolo261_custom/weights/last.pt, 53.0MB
Optimizer stripped from /content/drive/MyDrive/Project/weight/yolo261_custom/weights/best.pt, 53.0MB

Validating /content/drive/MyDrive/Project/weight/yolo261_custom/weights/best.pt...
Ultralytics 8.4.14 Python-3.12.12 torch-2.10.0+cu120 CUDA10 (Tesla T4, 14919MiB)
YOLOv26l summary (fused): 190 layers, 24,748,824 parameters, 0 gradients, 86.1 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.3it/s 0.8s
all 34 304 0.979 0.928 0.969 0.683
quantity_item 32 88 0.982 0.932 0.962 0.566
product_description 34 93 0.973 0.892 0.965 0.701
price 34 89 0.977 0.976 0.981 0.725
total_due_amount 34 34 0.985 0.912 0.967 0.74

Speed: 0.2ms preprocess, 15.5ms inference, 0.0ms loss, 0.3ms postprocess per image
Results saved to /content/drive/MyDrive/Project/weight/yolo261_custom
Learn more at https://docs.ultralytics.com/modes/train
```

Figure 5: Finished training model in Google Colab snapshot.

10. Model Evaluation & Results

The model's performance on the validation set was outstanding, proving that the custom Albumentations pipeline successfully expanded the dataset without confusing the model's spatial awareness.

The final evaluation yielded an impressive overall mAP50 of 96.9% (0.969).

Breaking down the precision by specific classes:

price: 98.1% mAP50

total_due_amount: 96.7% mAP50

product_description: 96.5% mAP50

quantity_item: 96.2% mAP50

Analysis: The price class achieved the highest accuracy, likely because prices on Thai receipts (especially 7-Eleven) are consistently formatted on the far-right edge of the paper. Despite the visual similarity between an individual price and the total_due_amount, the model successfully learned to differentiate them based on their vertical positioning (context), achieving over 96% accuracy for both.

Furthermore, the yolo26l (Large) architecture successfully detected the quantity_item targets (96.2%), which are often difficult for smaller models to catch since they are usually just a single, isolated digit.

11. Error Analysis & Training Metrics

To visually analyze where the model succeeds and where it makes false predictions, I evaluated both the absolute and normalized confusion matrices (confusion_matrix.png and confusion_matrix_normalized.png), alongside the training results graph generated during the validation phase.

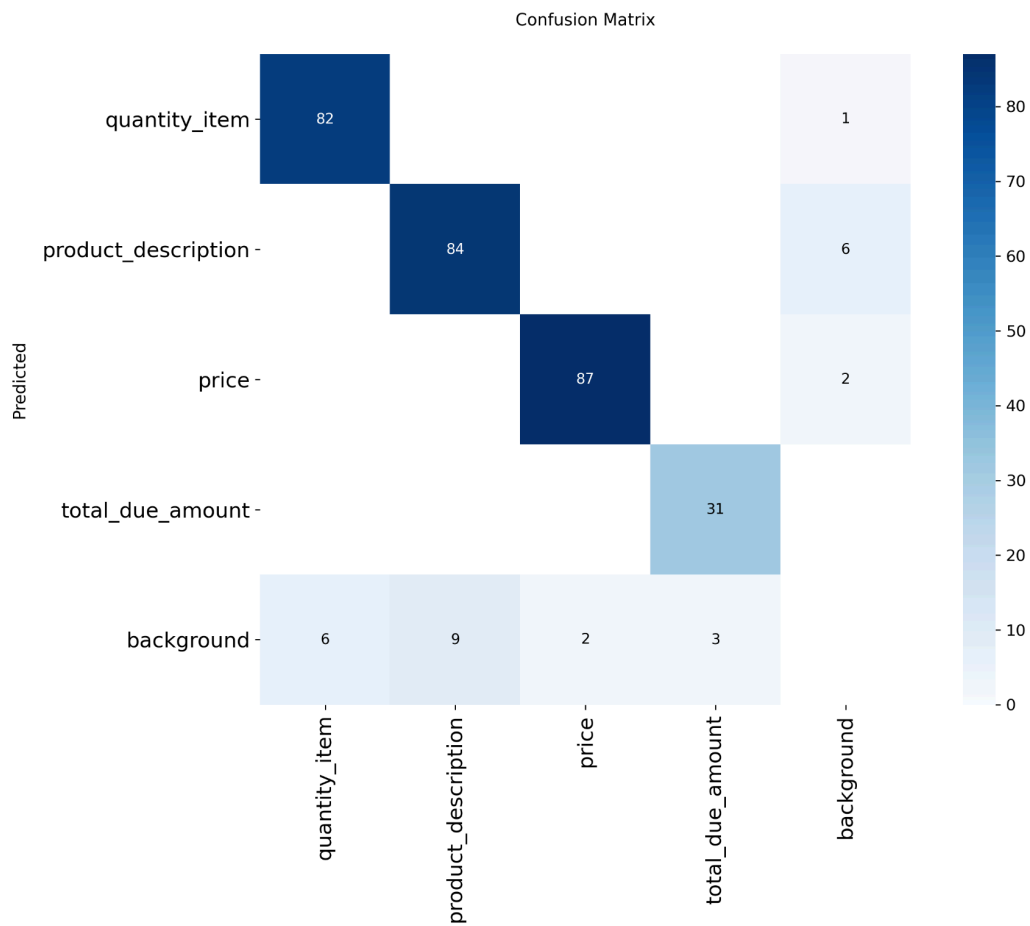


Figure 6: Absolute Confusion Matrix showing the exact count of predictions.

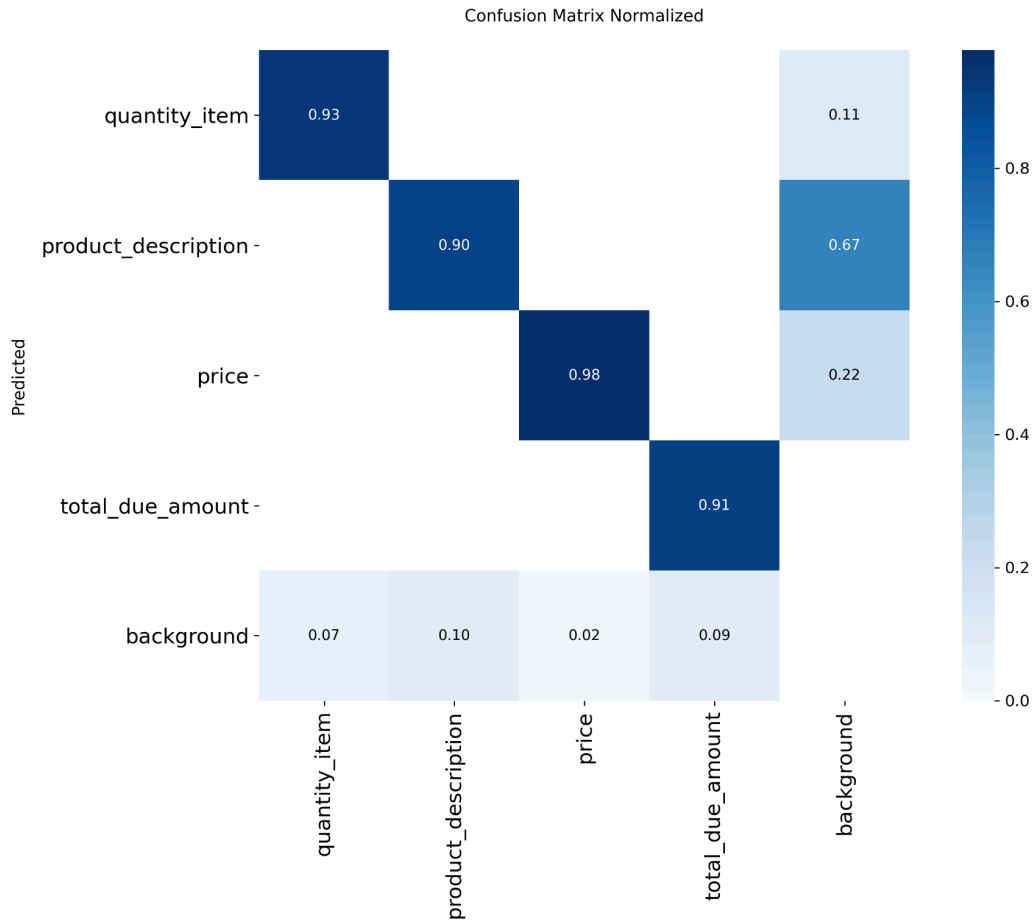


Figure 7: Normalized Confusion Matrix showing the percentage distribution of predictions.

Both matrices show a strong dark diagonal, signifying high True Positives across all four classes. Including both figures is key: the absolute matrix reveals an exceptionally low raw number of background false positives (predicting a class where there is only white space). This minimal background error rate demonstrates that the model's bounding boxes are very tight and accurate, which is crucial for clean handoff to an OCR engine.

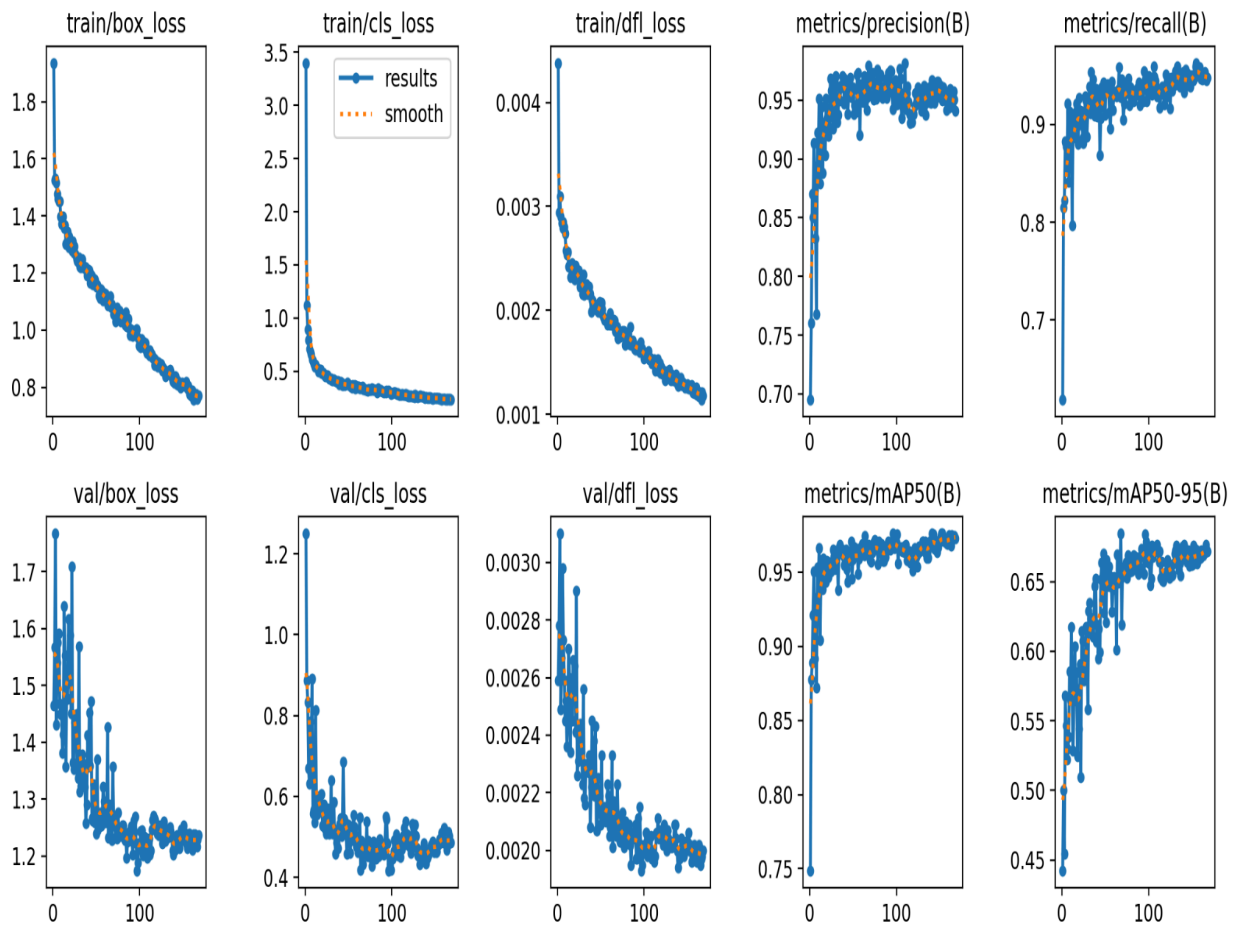


Figure 8: Training results showing a smooth, continuous decrease in bounding box and classification loss.

12. Limitations and Edge Case Analysis

While the model achieved a 96.9% mAP50 on the validation set, real-world inference testing revealed specific limitations when encountering out-of-distribution (OOD) data.

Test Case 1: Unseen Formats (Tops Supermarket - Figure 9)

When tested on a Tops receipt, the model struggled to detect the price and total_due_amount. Analysis of the receipt shows two primary causes:

Alphanumeric Noise: The price was printed as 69.00N. Because the model was trained strictly on numerical prices, the trailing alphabetical character disrupted detection.

Keyword Variation: The model learned to associate totals with the Thai keyword "ยอดสุทธิ" (Net Total). The Tops receipt used the English phrase "TOTAL(Qty) 1 Items," lacking the contextual anchor the model expected.

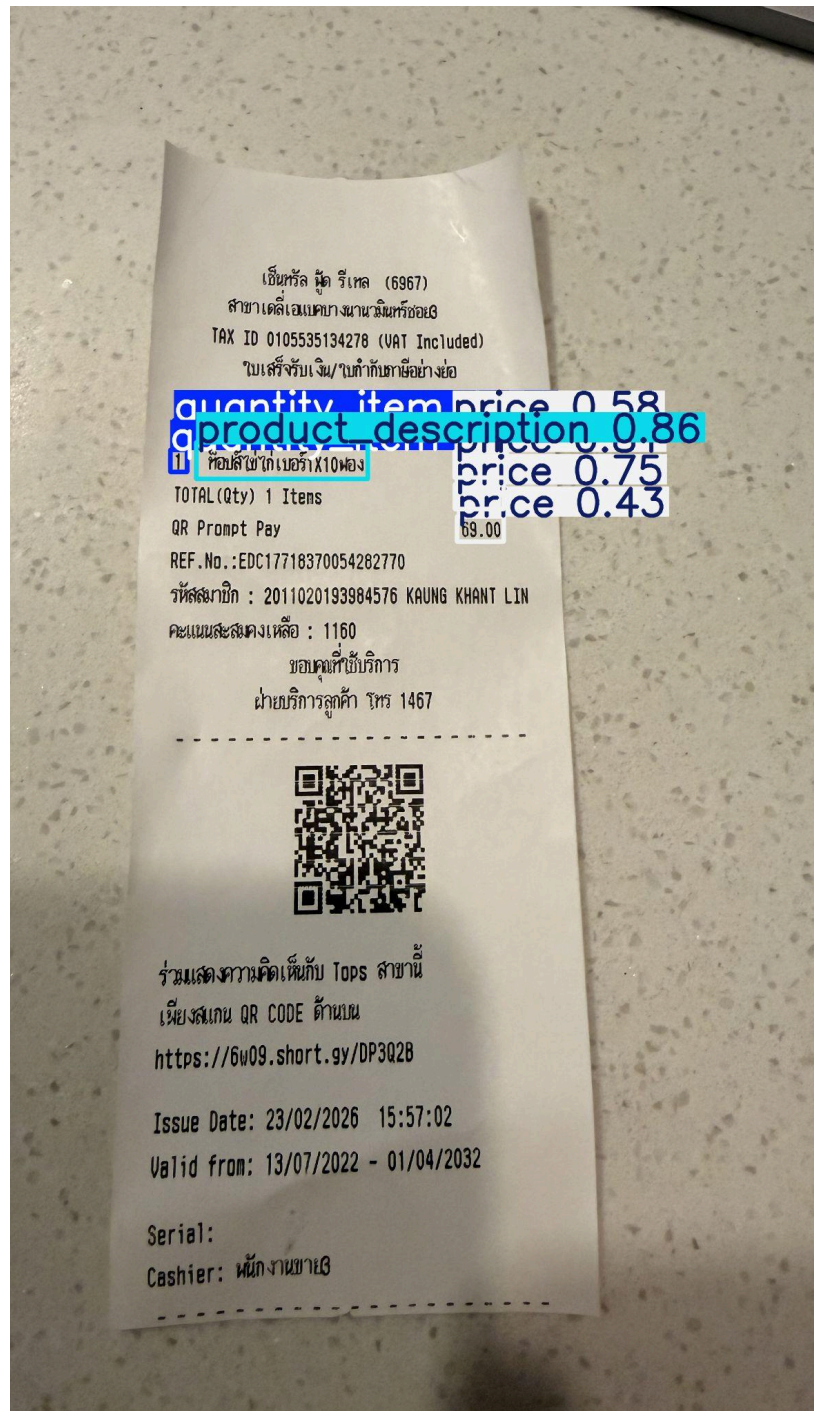
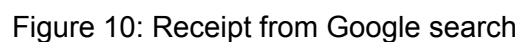


Figure 9: Tops receipt

When tested on an older, dot-matrix printed restaurant receipt, the model failed to capture the line items. The spatial geometry (extremely wide column gaps) and the pixelated, low-resolution font structure differed entirely from the thermal-printed receipts (like 7-Eleven and Uniqlo) that dominated the training distribution.



To resolve these edge cases, the next iteration of this project would require expanding the dataset from ~200 images to several thousand, specifically targeting a wider variance of retail layouts, bilingual receipts, and legacy printer fonts.

13. Final Output & Conclusion

Because the model triggered Early Stopping, the Ultralytics framework automatically stripped the optimizer states to reduce the file size (to 53.0MB) and saved the highest-performing checkpoint.

This final, ready-to-use model file [best.pt](#) was automatically exported to my Google Drive directory. By systematically collecting real-world data, manually establishing strict labeling rules in Labelme, applying geometrically-safe augmentations, and leveraging cloud-based transfer learning, I successfully built a highly accurate object detection model tailored for Thai tax invoices. This exact model file, alongside this detailed workflow report, serves as the final submission for this project.